

[Marcar como hecha](#)

12.3. Acciones JSP y directivas

Los elementos de acción especifican actividades que deben realizarse cuando se solicita la página, al igual que los elementos de scripting, ya que su propósito es precisamente encapsular actividades que Tomcat realiza al gestionar una petición HTTP de un cliente. Los elementos de acción pueden utilizar, modificar y/o crear objetos, y pueden afectar a la forma en que se envían los datos a la salida. Hay más de una docena de acciones estándar: `attribute`, `body`, `element`, `fallback`, `forward`, `getProperty`, `include`, `param`, `params`, `plugin`, `setProperty`, `text`, y `useBean`. Por ejemplo, el siguiente elemento de acción incluye en una página JSP la salida de otra página:

```
<jsp:include page="otra.jsp"/>
```

Además de los elementos de acción estándar, JSP también proporciona un mecanismo que permite definir acciones personalizadas, en las que un prefijo de su elección sustituye al prefijo `jsp` de las acciones estándar. El mecanismo de extensión de etiquetas permite crear bibliotecas de acciones personalizadas que podrá utilizar en todas sus aplicaciones. Varias acciones personalizadas se hicieron tan populares entre la comunidad de programadores que Sun Microsystems (ahora Oracle) decidió estandarizarlas. El resultado es JSTL, la biblioteca de etiquetas estándar de JSP.

Elementos de secuencia de comandos y Java

Los elementos de script permiten incrustar código Java en una página HTML. Todos los ejecutables Java ya sea un programa independiente que se ejecuta directamente en un entorno de tiempo de ejecución o un servlet que se ejecuta en un contenedor como Tomcat- se reduce a la instanciación de clases en objetos y ejecutar sus métodos. Esto puede no ser tan evidente con JSP, ya que Tomcat envuelve cada página JSP en una clase de tipo Servlet entre bastidores, pero sigue siendo se aplica.

Un método Java consiste en una secuencia de operaciones -ejecutadas cuando el método es llamado con el alcance de instanciar objetos, asignar memoria para variables, calcular expresiones, realizar asignaciones o ejecutar otras instrucciones específicas.

Scriptlets

Un scriptlet es un bloque de código Java encerrado entre `<% y %>`. Por ejemplo, este código incluye dos scriptlets que permiten activar o desactivar un elemento HTML en función de una condición

```
<% if (condition) { %>
<p>This is only shown if the condition is satisfied</p>
<% } %>
```

Expresiones

Un elemento de script de expresión inserta en la página el resultado de una expresión Java encerrada en el par `<%= y %>`. Por ejemplo, en el siguiente fragmento de código, el elemento de secuencia de comandos de expresión inserta la fecha actual en la página HTML generada:

```
<%@page import="java.util.Date"%>
Server date and time: <%=new Date()%>
```

Puede utilizar dentro de un elemento de script de expresión cualquier expresión Java, siempre que dé como resultado un valor. En la práctica, esto significa que cualquier expresión Java servirá, excepto la ejecución de un método de tipo void. Por ejemplo, `<%= (condición) ? "sí" : "no"%>` es válido, porque calcula una cadena. Obtendría la misma salida con el scriptlet `<%if (condición) out.print("sí") else out.print("no");%>`. Tenga en cuenta que una expresión no es una sentencia Java. Por lo tanto, no tiene punto y coma al final.

Declaraciones

Un elemento de script de declaración es una declaración de variable Java encerrada entre `<%! y %>`. Da lugar a una variable de instancia compartida por todas las solicitudes de la misma página.

Así como la palabra clave final, hace que la variable sea inmutable. Del modo como se definen las constantes en Java, también la palabra clave static

indica que una variable debe ser compartida por todos los objetos dentro de la misma aplicación que son instanciados a partir de la clase.

El uso de variables estáticas en JSP requiere algún comentario adicional. En JSP, puede declarar variables de tres maneras:

```
<% int k = 0; %>
```

```
<%! int k = 0; %>
```

```
<%! static int k = 0; %>
```

La primera declaración significa que se crea una nueva variable para cada solicitud entrante del cliente HTTP entrante; la segunda significa que se crea una nueva variable para cada nueva instancia del servlet; y la tercera significa que la variable se comparte entre todas las instancias del servlet.

Objetos de Aplicación

ejemplo práctico:

Crear una aplicación web con dos jsp

hazlo.jsp:

```
<%@page language="java" contentType="text/html"%>
<html> <head> <title>Código condicional ON</title> </head>
<body>Condición
<%
    application.setAttribute("hazlo", "");
    if (application.getAttribute("hazlo") == null) {
        out.print("no lo hagas");
    }else{
        out.print("hazlo");
    }
    %>
habilitada</body> </html>
```

no_hagas.jsp

```
<%@page language="java" contentType="text/html"%>
<html> <head> <title>Código condicional OFF</title> </head>
<body>Condición
<%
    application.removeAttribute("hazlo");
    if (application.getAttribute("hazlo") == null) {
        out.print("no lo hagas");
    }else{
        out.print("hazlo");
    }
    %>
habilitada</body> </html>
```

luego se ejecuta y se llama a cada página:

el resultado será el el primer caso

Condición hazlo habilitada

y en el segundo

Condición no lo hagas habilitada

Objeto exception, casos prácticos 2

cause_exception.jsp;

```
<%@page language="java" contentType="text/html"%>
<%@page errorPage="stack_trace.jsp"%>
<html> <head> <title>Cause null pointer exception</title> </head> <body>
<%
```

?

```
String a = request.getParameter("notThere");
int len = a.length(); // causes a null pointer exception
%>
</body> </html>
```

stack_trace.jsp:

```
<%@page language="java" contentType="text/html"%>
<%@page import="java.util.*, java.io.*"%>
<%@page isErrorPage="true"%>
<html> <head> <title>Print stack trace</title> </head> <body>
From exception.getStackTrace():<br/>
<pre><%
    StackTraceElement[] trace = exception.getStackTrace();
    for (int k = 0; k < trace.length; k++) {
        out.println(trace[k]);
    }
%></pre>
Printed with exception.printStackTrace(new PrintWriter(out)):
<pre><%
    exception.printStackTrace(new PrintWriter(out));
%></pre>
</body> </html>
```

la primera lanza:

From exception.getStackTrace():

```
org.apache.jsp.cause_005fexception_jsp._jspService(cause_005fexception_jsp.java:123)
org.apache.jasper.runtime.HttpJspBase.service(HttpJspBase.java:70)
javax.servlet.http.HttpServlet.service(HttpServlet.java:764)
org.apache.jasper.servlet.JspServletWrapper.service(JspServletWrapper.java:466)
org.apache.jasper.servlet.JspServlet.serviceJspFile(JspServlet.java:379)
org.apache.jasper.servlet.JspServlet.service(JspServlet.java:327)
javax.servlet.http.HttpServlet.service(HttpServlet.java:764)
org.apache.catalina.core.ApplicationFilterChain.internalDoFilter(ApplicationFilterChain.java:227)
org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:162)
org.apache.tomcat.websocket.server.WsFilter.doFilter(WsFilter.java:53)
org.apache.catalina.core.ApplicationFilterChain.internalDoFilter(ApplicationFilterChain.java:189)
org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:162)
org.apache.catalina.core.StandardWrapperValve.invoke(StandardWrapperValve.java:197)
org.apache.catalina.core.StandardContextValve.invoke(StandardContextValve.java:97)
org.apache.catalina.authenticator.AuthenticatorBase.invoke(AuthenticatorBase.java:541)
org.apache.catalina.core.StandardHostValve.invoke(StandardHostValve.java:135)
org.apache.catalina.valves.ErrorReportValve.invoke(ErrorReportValve.java:92)
org.apache.catalina.valves.AbstractAccessLogValve.invoke(AbstractAccessLogValve.java:687)
org.apache.catalina.core.StandardEngineValve.invoke(StandardEngineValve.java:78)
org.apache.catalina.connector.CoyoteAdapter.service(CoyoteAdapter.java:360)
org.apache.coyote.http11.Http11Processor.service(Http11Processor.java:399)
org.apache.coyote.AbstractProcessorLight.process(AbstractProcessorLight.java:65)
org.apache.coyote.AbstractProtocol$ConnectionHandler.process(AbstractProtocol.java:889)
org.apache.tomcat.util.net.NioEndpoint$SocketProcessor.doRun(NioEndpoint.java:1743)
org.apache.tomcat.util.net.SocketProcessorBase.run(SocketProcessorBase.java:49)
org.apache.tomcat.util.threads.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1191)
org.apache.tomcat.util.threads.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:659)
org.apache.tomcat.util.threads.TaskThread$WrappingRunnable.run(TaskThread.java:61)
java.base/java.lang.Thread.run(Thread.java:833)
```

Printed with exception.printStackTrace(new PrintWriter(out)):

```
java.lang.NullPointerException: Cannot invoke "String.length()" because "a" is null
    at org.apache.jsp.cause_005fexception_jsp._jspService(cause_005fexception_jsp.java:123)
    at org.apache.jasper.runtime.HttpJspBase.service(HttpJspBase.java:70)
    at javax.servlet.http.HttpServlet.service(HttpServlet.java:764)
    at org.apache.jasper.servlet.JspServletWrapper.service(JspServletWrapper.java:466)
    at org.apache.jasper.servlet.JspServlet.serviceJspFile(JspServlet.java:379)
    at org.apache.jasper.servlet.JspServlet.service(JspServlet.java:327)
    at javax.servlet.http.HttpServlet.service(HttpServlet.java:764)
    at org.apache.catalina.core.ApplicationFilterChain.internalDoFilter(ApplicationFilterChain.java:227)
    at org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:162)
    at org.apache.tomcat.websocket.server.WsFilter.doFilter(WsFilter.java:53)
    at org.apache.catalina.core.ApplicationFilterChain.internalDoFilter(ApplicationFilterChain.java:189)
    at org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:162)
    at org.apache.catalina.core.StandardWrapperValve.invoke(StandardWrapperValve.java:197)
    at org.apache.catalina.core.StandardContextValve.invoke(StandardContextValve.java:97)
    at org.apache.catalina.authenticator.AuthenticatorBase.invoke(AuthenticatorBase.java:541)
    at org.apache.catalina.core.StandardHostValve.invoke(StandardHostValve.java:135)
    at org.apache.catalina.valves.ErrorReportValve.invoke(ErrorReportValve.java:92)
    at org.apache.catalina.valves.AbstractAccessLogValve.invoke(AbstractAccessLogValve.java:687)
    at org.apache.catalina.core.StandardEngineValve.invoke(StandardEngineValve.java:78)
    at org.apache.catalina.connector.CoyoteAdapter.service(CoyoteAdapter.java:360)
    at org.apache.coyote.http11.Http11Processor.service(Http11Processor.java:399)
    at org.apache.coyote.AbstractProcessorLight.process(AbstractProcessorLight.java:65)
    at org.apache.coyote.AbstractProtocol$ConnectionHandler.process(AbstractProtocol.java:889)
    at org.apache.tomcat.util.net.NioEndpoint$SocketProcessor.doRun(NioEndpoint.java:1743)
    at org.apache.tomcat.util.net.SocketProcessorBase.run(SocketProcessorBase.java:49)
    at org.apache.tomcat.util.threads.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1191)
    at org.apache.tomcat.util.threads.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:659)
    at org.apache.tomcat.util.threads.TaskThread$WrappingRunnable.run(TaskThread.java:61)
    at java.base/java.lang.Thread.run(Thread.java:833)
```

la segunda

HTTP Status 500 – Internal Server Error

Type Exception Report

Message Ha sucedido una excepción al procesar la página JSP [/stack_trace.jsp] en línea [13]

Description The server encountered an unexpected condition that prevented it from fulfilling the request.

Exception

```
org.apache.jasper.JasperException: Ha sucedido una excepción al procesar la página JSP [/stack_trace.jsp] en línea [13]

10: <html><head><title>Print stack trace</title></head><body>
11: From exception.getStackTrace():<br/>
12: <pre><%
13:   StackTraceElement[] trace = exception.getStackTrace();
14:   for (int k = 0; k < trace.length; k++) {
15:     out.println(trace[k]);
16:   }

Stacktrace:

    org.apache.jasper.servlet.JspServletWrapper.handleJspException(JspServletWrapper.java:610)
    org.apache.jasper.servlet.JspServletWrapper.service(JspServletWrapper.java:499)
    org.apache.jasper.servlet.JspServlet.serviceJspFile(JspServlet.java:379)
    org.apache.jasper.servlet.JspServlet.service(JspServlet.java:327)
    javax.servlet.http.HttpServlet.service(HttpServlet.java:764)
    org.apache.tomcat.websocket.server.WsFilter.doFilter(WsFilter.java:53)
```

Root Cause

?

```
java.lang.NullPointerException: Cannot invoke "java.lang.Throwable.getStackTrace()" because "exception" is null
    org.apache.jsp.stack_005ftrace_jsp._jspService(stack_005ftrace_jsp.java:120)
    org.apache.jasper.runtime.HttpJspBase.service(HttpJspBase.java:70)
    javax.servlet.http.HttpServlet.service(HttpServlet.java:764)
    org.apache.jasper.servlet.JspServletWrapper.service(JspServletWrapper.java:466)
    org.apache.jasper.servlet.JspServlet.serviceJspFile(JspServlet.java:379)
    org.apache.jasper.servlet.JspServlet.service(JspServlet.java:327)
    javax.servlet.http.HttpServlet.service(HttpServlet.java:764)
    org.apache.tomcat.websocket.server.WsFilter.doFilter(WsFilter.java:53)
```

Note The full stack trace of the root cause is available in the server logs.

Apache Tomcat/9.0.60

Última modificación: martes, 14 de marzo de 2023, 13:35